Ingeniería en Informática y Sistemas
Manejo e Implementación de Archivos

Laboratorio No. 2

Agenda de Contactos con Acceso Aleatorio

Registros de tamaño fijo, seek() / tell(), índice primario en JSON, exportación JSON y PDF, integridad SHA-256 y respaldo ZIP

Introducción

Este documento describe la aplicación de escritorio **Agenda de Contactos con Acceso Aleatorio**, desarrollada en Python con una interfaz gráfica en Tkinter. La agenda guarda los contactos en el archivo **contactos.dat**, donde todos los registros miden exactamente lo mismo. Gracias a eso, la aplicación llega directamente a cualquier contacto con **seek()** sin leer el archivo completo, y se apoya en un índice guardado en **indice.json** para las búsquedas por id.

Además, la aplicación exporta los contactos a JSON y genera un reporte PDF a partir de ese JSON, verifica la integridad del archivo con SHA-256, crea respaldos comprimidos en ZIP y muestra los metadatos y permisos del archivo. Todas las capturas de este documento se tomaron ejecutando la aplicación.

Organización del código

Archivo	Función
main.py	Punto de entrada de la aplicación.
config.py	Rutas de los archivos y estructura del registro (largo de cada campo).
registros.py	Registros de tamaño fijo: lectura, escritura y modificación con seek() y tell().
indice.py	Índice primario en indice.json (id -> número de registro).
agenda.py	Agregar, buscar por id y modificar, con validaciones y manejo de errores.
integridad.py	Cálculo y verificación del hash SHA-256.
exportar.py	Exportación a JSON y reporte PDF (reportlab).
herramientas.py	Respaldo ZIP e información del archivo (os.stat y os.access).
interfaz.py	Ventana principal, menú, formulario y diálogos.

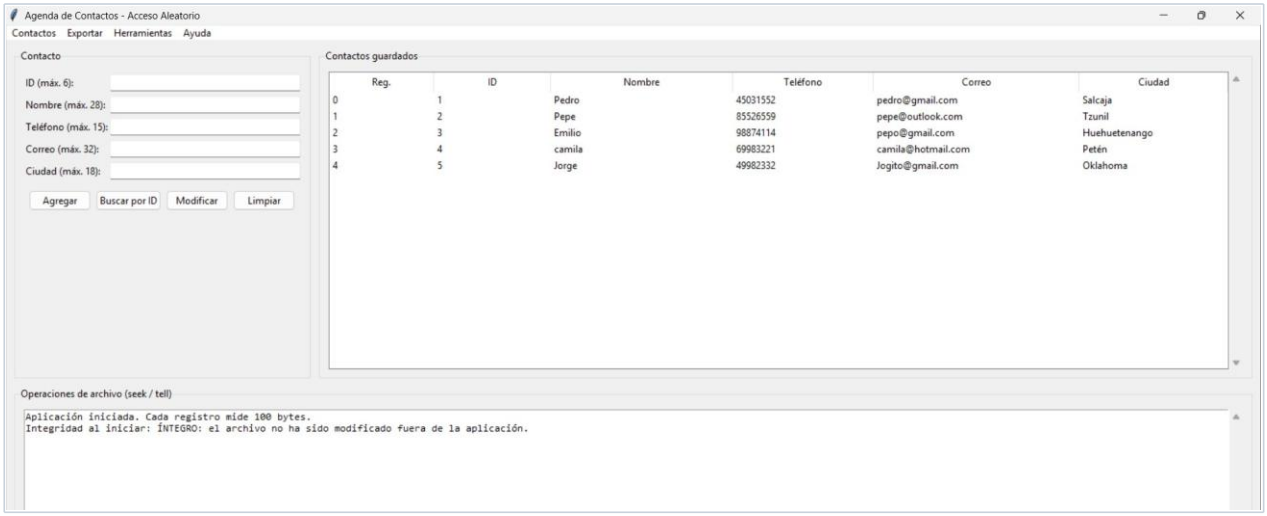


Figura 1. Ventana principal con los contactos guardados en contactos.dat.

1. Estructura del registro

Cada contacto se guarda como un registro de **100 bytes**. Cada campo tiene un largo máximo fijo; si el dato es más corto, se rellena con espacios usando **ljust()** hasta completar su largo. Al final de cada registro va un salto de línea para que el archivo se pueda abrir en un editor de texto.

Campo	Largo (bytes)	Posición dentro del registro	Ejemplo
id	6	0 - 5	2
nombre	28	6 - 33	Pepe
telefono	15	34 - 48	85526559
correo	32	49 - 80	pepe@outlook.com
ciudad	18	81 - 98	Tzunil
salto de línea	1	99	\n
Total	**100**		

El largo se mide en **bytes** y no en caracteres porque en UTF-8 una letra con tilde o una ñ ocupa 2 bytes. Por eso el archivo se abre en modo binario ("rb", "ab", "rb+"): así cada registro mide siempre 100 bytes, incluso en Windows, donde el modo texto convertiría el salto de línea en dos bytes. Como todos miden lo mismo, la posición de cualquier registro se calcula con una multiplicación:

```
posición = número de registro × 100

registro 0 -> byte 0      registro 1 -> byte 100      registro 5 -> byte 500
```

La estructura se define en **config.py**:

```
CAMPOS = [
    ("id", 6),
    ("nombre", 28),
    ("telefono", 15),
    ("correo", 32),
    ("ciudad", 18),
]

# Suma de los largos de los campos + 1 byte del salto de línea
TAMANO_REGISTRO = sum(largo for campo, largo in CAMPOS) + 1      # 100 bytes
```

Y cada registro se arma en **registros.py**, rellenando cada campo con espacios:

```
def formatear_campo(texto, largo):
    # Convierte el texto a bytes UTF-8 y lo rellena con espacios hasta su largo.
    texto = str(texto).strip().replace("\n", " ").replace("\r", " ")
    datos = texto.encode(CODIFICACION)
    while len(datos) > largo:          # si es muy largo, se recorta
        texto = texto[:-1]
        datos = texto.encode(CODIFICACION)
    return datos.ljust(largo, b" ")


def crear_registro(contacto):
    registro = b""
    for campo, largo in CAMPOS:
        registro += formatear_campo(contacto[campo], largo)
    registro += b"\n"
    return registro                    # siempre 100 bytes
```

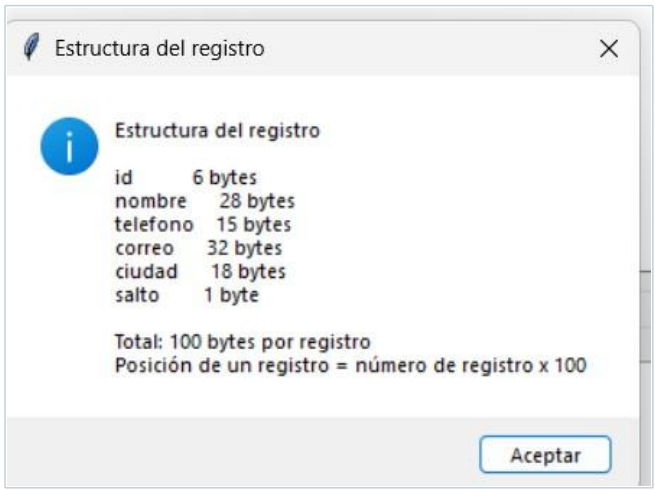


Figura 2. Opción Ayuda > Estructura del registro dentro de la aplicación.

2. Acceso aleatorio con seek() y tell()

Para leer o modificar un contacto, la aplicación no recorre el archivo: calcula la posición del registro (número de registro × 100) y usa **seek()** para moverse directamente ahí. Después usa **tell()** para mostrar en qué byte del archivo se encuentra. Cada operación escribe ese recorrido en el panel *Operaciones de archivo (seek / tell)* de la ventana.

2.1 Leer un registro

```
def leer_registro(numero):
    posicion = numero * TAMANO_REGISTRO

    with open(ARCHIVO_CONTACTOS, "rb") as archivo:
        archivo.seek(posicion)                  # salta directo al registro
        posicion_inicio = archivo.tell()        # confirma la posición

        datos = archivo.read(TAMANO_REGISTRO)   # lee solo 100 bytes
        posicion_final = archivo.tell()

    contacto = separar_registro(datos)
    return contacto, movimiento
```

2.2 Agregar un registro

Para agregar, el archivo se abre en modo "ab" y se usa **seek(0, 2)** para ir al final. Con **tell()** se obtiene el byte donde comenzará el nuevo registro; dividiéndolo entre 100 se conoce su número de registro, que luego se guarda en el índice.

```
with open(ARCHIVO_CONTACTOS, "ab") as archivo:
    archivo.seek(0, 2)                  # final del archivo
    posicion = archivo.tell()           # ej. 500

    if posicion % TAMANO_REGISTRO != 0:
        raise ValueError("El archivo de contactos no tiene registros completos.")

    numero = posicion // TAMANO_REGISTRO    # 500 // 100 = registro #5
    archivo.write(registro)
```

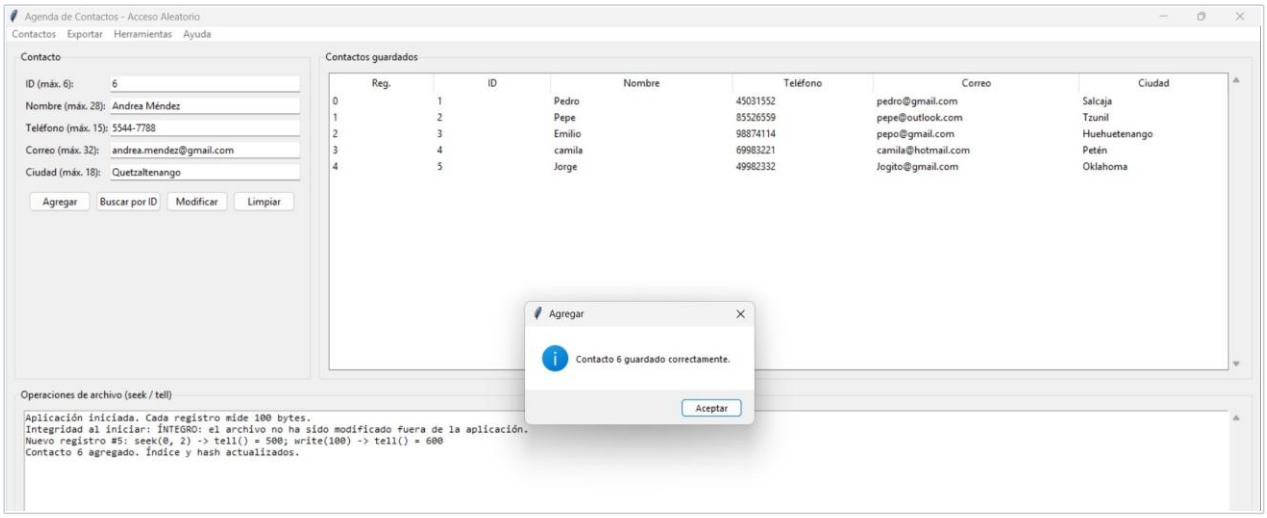


Figura 3. Al agregar el contacto 6, la bitácora muestra seek(0, 2) -> tell() = 500 y write(100) -> tell() = 600: el registro nuevo ocupa los bytes 500 a 599.

2.3 Modificación en el lugar

Al modificar, el archivo se abre en modo **"rb+"** (lectura y escritura sin borrar el contenido). Se hace seek() al inicio del registro y se escriben exactamente 100 bytes. Así solo cambia ese registro y el resto del archivo no se reescribe.

```
def modificar_registro(numero, contacto):
    registro = crear_registro(contacto)
    posicion = numero * TAMANO_REGISTRO

    with open(ARCHIVO_CONTACTOS, "rb+") as archivo:
        archivo.seek(posicion)
        posicion_inicio = archivo.tell()
        archivo.write(registro)              # sobrescribe solo 100 bytes
        posicion_final = archivo.tell()
```

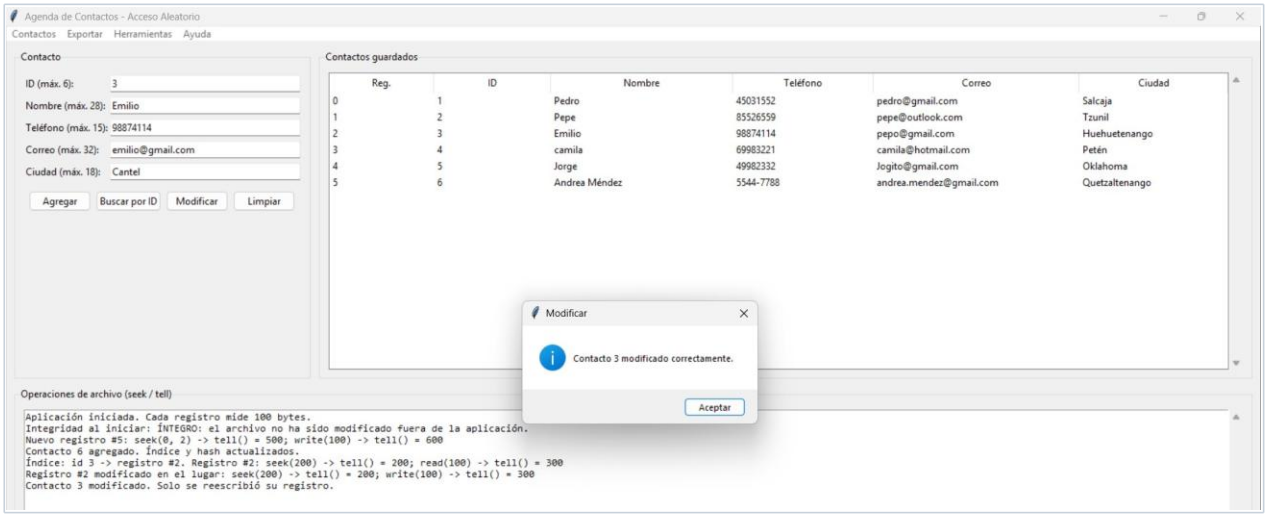


Figura 4. Modificación del contacto 3 (ciudad y correo): seek(200) -> tell() = 200 y write(100) -> tell() = 300. Solo se reescribió el registro #2.

3. Índice primario

El archivo **indice.json** relaciona cada **id** (clave) con su **número de registro** (valor) dentro de contactos.dat. Al iniciar, la aplicación lo carga en un diccionario de Python. Este es el índice después de agregar el contacto 6:


```
{
      "1":   0,
      "2":   1,
      "3":   2,
      "4":   3,
      "5":   4,
      "6":   5
}
```


3.1 Búsqueda por id

La búsqueda consulta primero el índice para obtener el número de registro y luego hace seek() a ese registro. Si el id no está en el índice, avisa que no existe sin abrir el archivo.

```
def buscar(self, id_texto):
    id_contacto = self.validar_id(id_texto)

    # Primero se consulta el índice
    if id_contacto not in self.indice:
        raise ErrorAgenda(f"No existe ningún contacto con el id {id_contacto}.")

    numero = self.indice[id_contacto]

    # Luego se hace seek() directo al registro
    contacto, movimiento = leer_registro(numero)
```

El índice se actualiza cada vez que se agrega un contacto (se añade la pareja id -> número de registro y se vuelve a guardar indice.json). Si indice.json no existe, está corrupto o no coincide con la cantidad de registros, la aplicación lo reconstruye recorriendo contactos.dat.

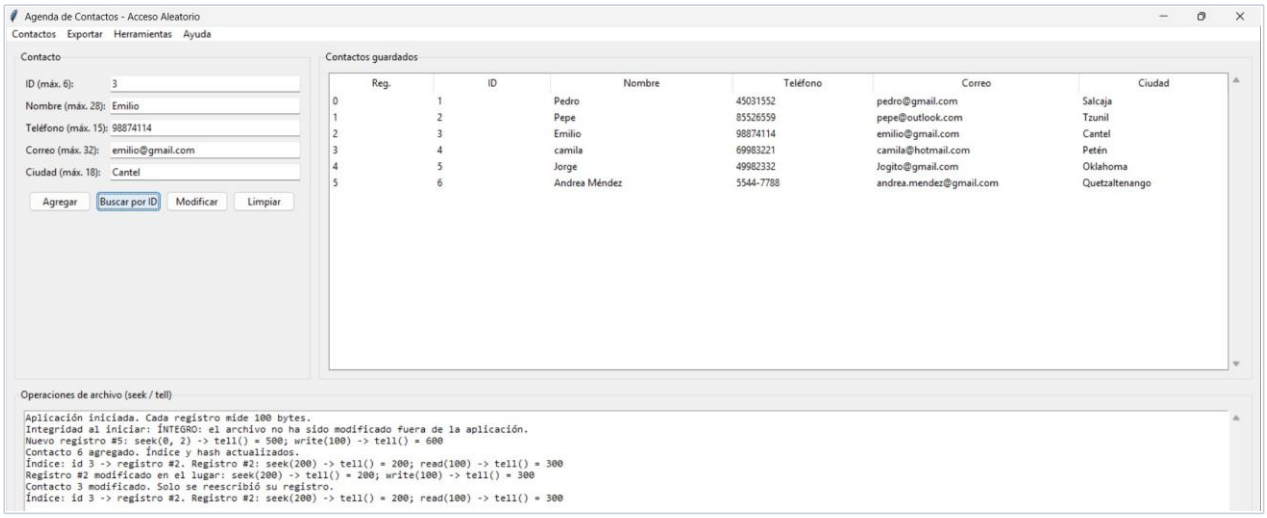


Figura 5. Búsqueda del id 3: el índice indica el registro #2, se hace seek(200) y se leen 100 bytes (tell() pasa de 200 a 300).

3.2 ¿Por qué es más eficiente que una búsqueda secuencial?

En una **búsqueda secuencial** hay que leer el archivo desde el inicio, registro por registro, comparando el id de cada uno hasta encontrarlo. Si el contacto está al final, o si no existe, se termina leyendo el archivo completo. Con el **índice**, la ubicación se obtiene del diccionario en memoria y solo se lee del disco el registro buscado: siempre **una sola lectura de 100 bytes**, sin importar el tamaño del archivo.

Cantidad de contactos	Secuencial (promedio)	Secuencial (peor caso / id inexistente)	Con índice
6	3 registros	6 registros	1 registro
1,000	500 registros	1,000 registros	1 registro
100,000	50,000 registros	100,000 registros	1 registro

Además, cuando el id no existe, el índice lo detecta sin leer el archivo, mientras que la búsqueda secuencial tendría que recorrerlo completo para saberlo. Esto solo es posible porque los registros son de tamaño fijo: con el número de registro se calcula la posición exacta.

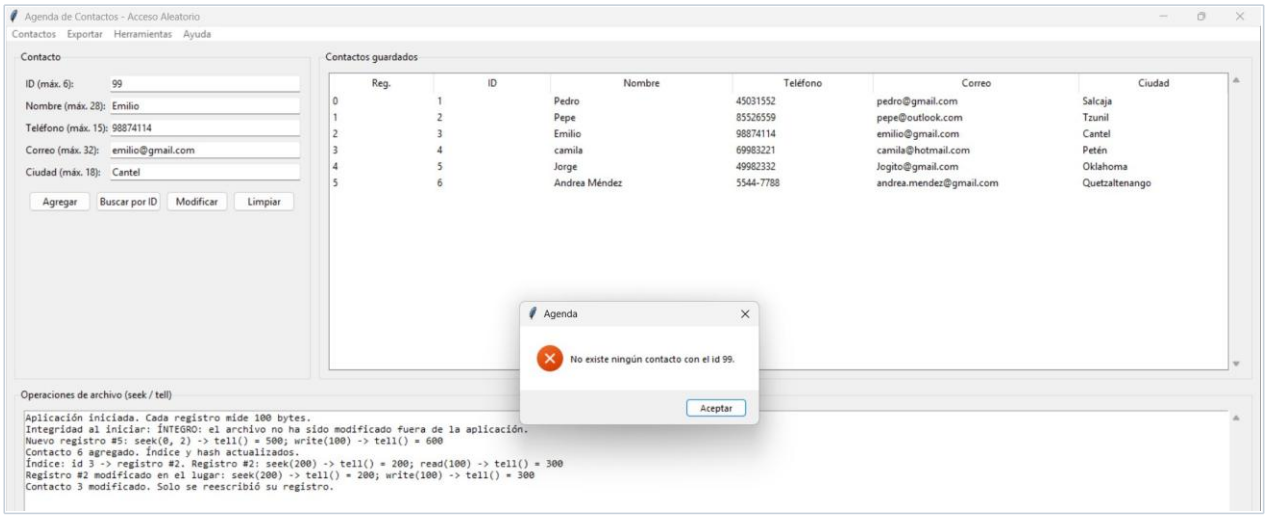


Figura 6. Búsqueda de un id que no existe (99): la aplicación avisa sin cerrarse.

4. Verificación de integridad con SHA-256

Cada vez que se guarda un cambio (agregar o modificar), la aplicación calcula el hash SHA-256 de contactos.dat con el módulo **hashlib** y lo guarda en **contactos.sha256**. Al iniciar, lo recalcula y lo compara con el guardado. Si no coinciden, avisa que el archivo fue modificado fuera de la aplicación. El hash se calcula leyendo el archivo por bloques:

```
def calcular_hash(ruta):
    sha256 = hashlib.sha256()
    with open(ruta, "rb") as archivo:
        while True:
            bloque = archivo.read(4096)
            if not bloque:
                break
            sha256.update(bloque)
    return sha256.hexdigest()
```

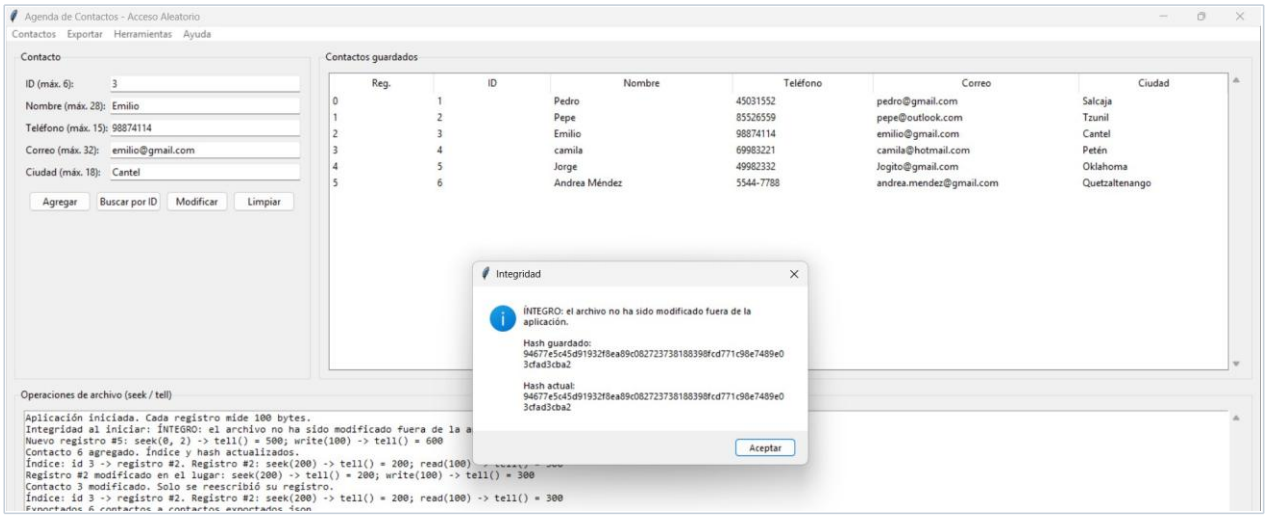


Figura 7. Herramientas > Verificar integridad con el archivo sin alterar: los dos hashes coinciden (ÍNTEGRO).

4.1 Prueba con un archivo alterado manualmente

Con la aplicación cerrada, se abrió contactos.dat en el Bloc de notas y se cambió a mano el nombre **Pedro** por **Pablo** en el primer registro. El cambio mantiene el mismo largo, así que el archivo sigue midiendo 600 bytes y la estructura parece correcta; aun así, el contenido ya no es el mismo.

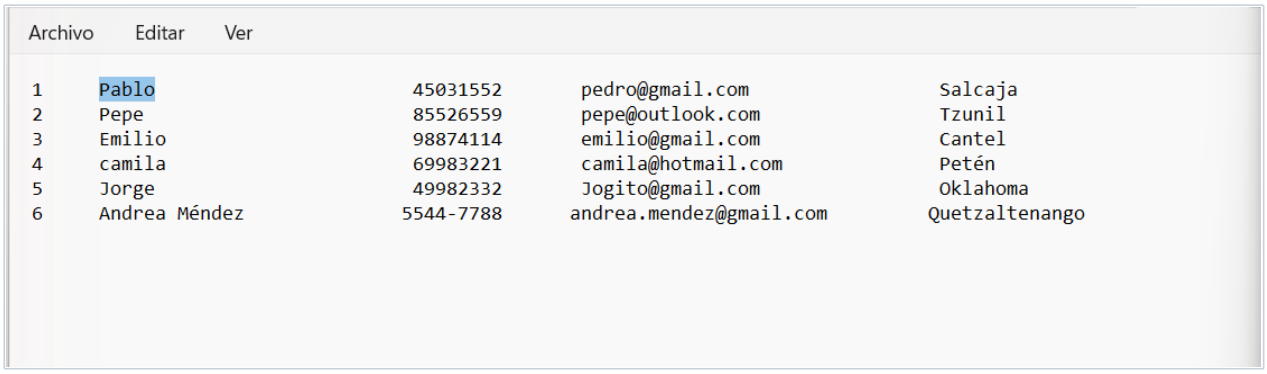


Figura 8. contactos.dat editado a mano en el Bloc de notas: 'Pedro' cambiado por 'Pablo'.

Al volver a abrir la aplicación, el hash recalculado ya no coincide con el guardado y se muestra la alerta con ambos hashes. Se respondió **No** para no aceptar el archivo alterado como válido.

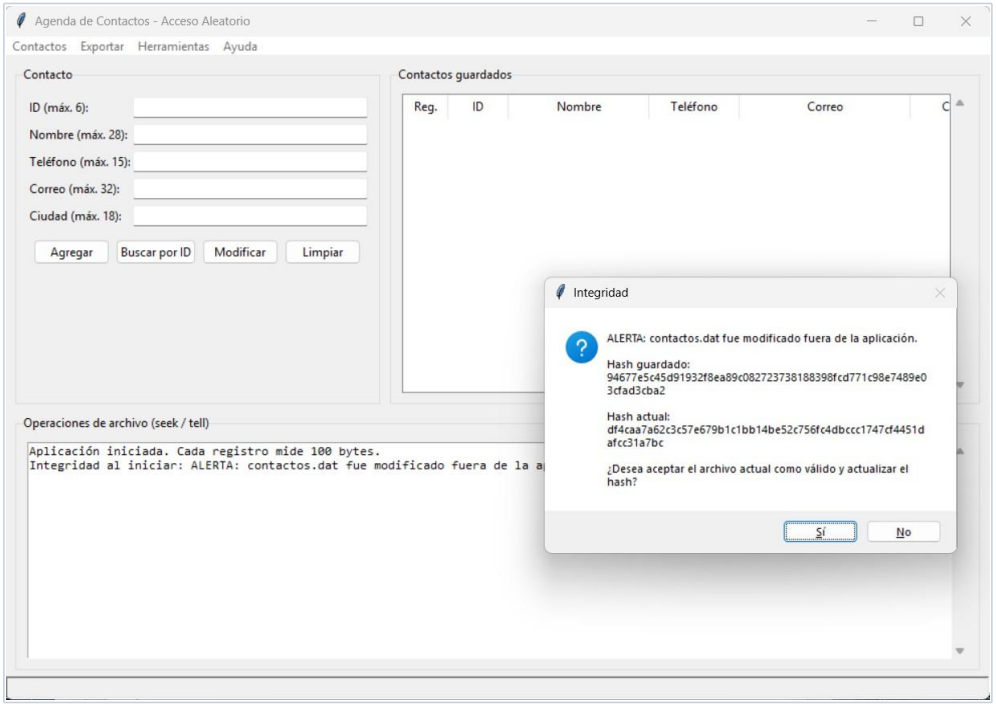


Figura 9. Alerta al iniciar: contactos.dat fue modificado fuera de la aplicación. Hash guardado 94677e5c... y hash actual df4caa7a...

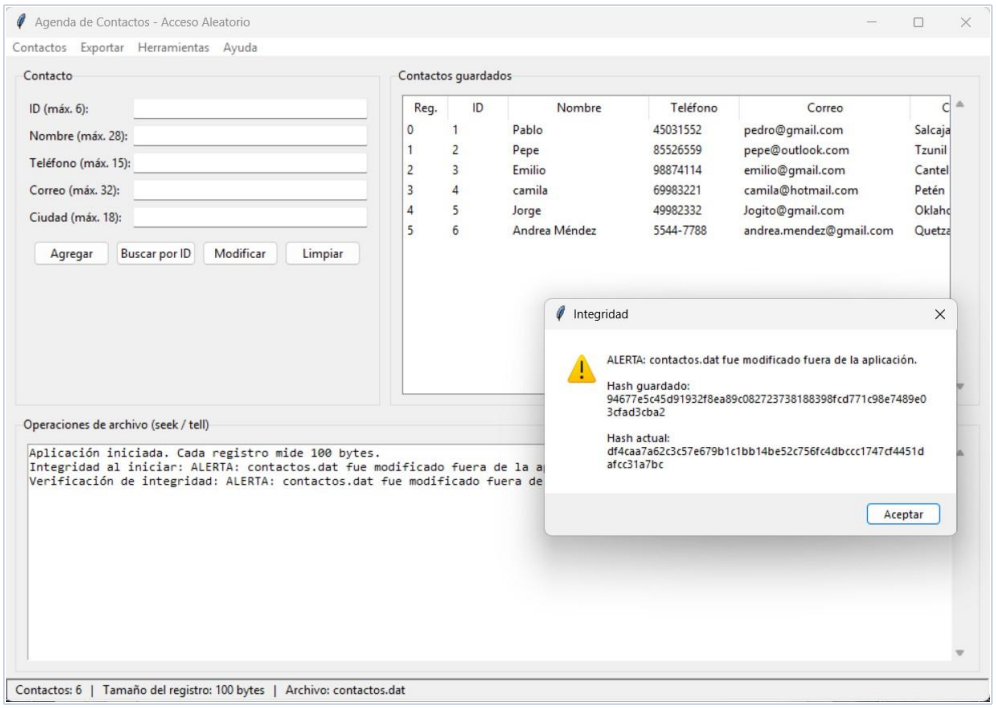


Figura 10. La verificación manual desde Herramientas también detecta la alteración; en la lista ya aparece 'Pablo'.

Un cambio de una sola palabra produce un hash totalmente distinto, por eso cualquier modificación externa se detecta aunque el tamaño del archivo no cambie.

5. Respaldo comprimido en ZIP

La opción Herramientas > Crear respaldo ZIP usa el módulo **zipfile** con compresión **ZIP_DEFLATED** para guardar contactos.dat e indice.json en la carpeta datos/respaldos. El nombre del archivo lleva la fecha y hora para no reemplazar respaldos anteriores. Al terminar se muestra el tamaño original, el comprimido y el porcentaje de reducción.

```
with zipfile.ZipFile(ruta_zip, "w", compression=zipfile.ZIP_DEFLATED) as archivo_zip:
    for ruta in [ARCHIVO_CONTACTOS, ARCHIVO_INDICE]:
        archivo_zip.write(ruta, arcname=ruta.name)
        tamano_original += os.path.getsize(ruta)

tamano_zip = os.path.getsize(ruta_zip)
```

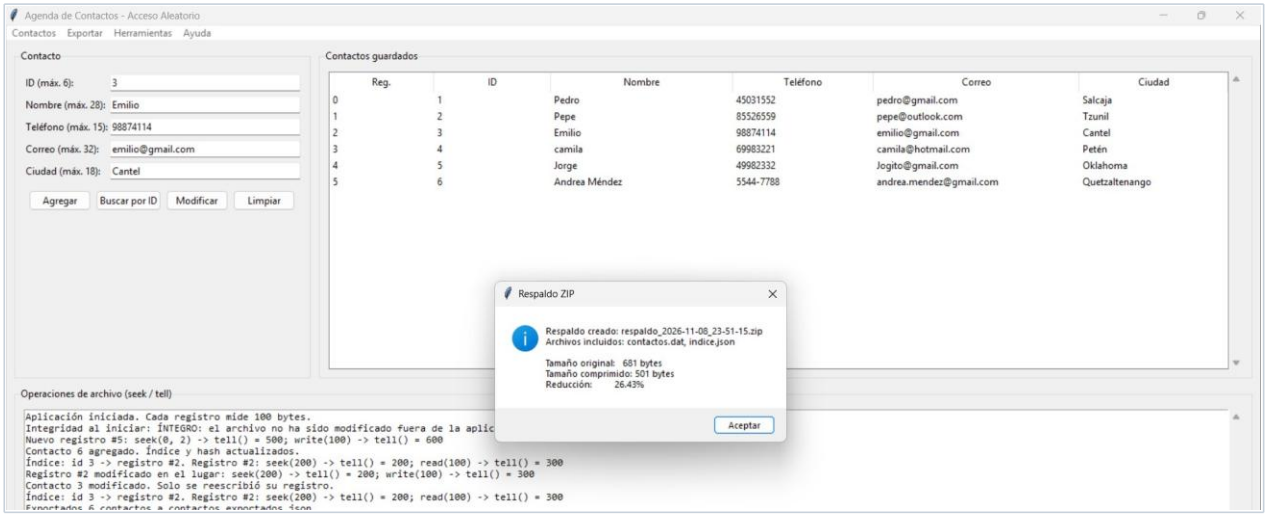


Figura 11. Respaldo creado: respaldo_2026-11-08_23-51-15.zip.

Comparación de tamaños

Archivo	Tamaño
contactos.dat (6 registros × 100 bytes)	600 bytes
indice.json	81 bytes
Total original	**681 bytes**
respaldo ZIP	**501 bytes**
Reducción	**26.43 %** (180 bytes menos)

La reducción es moderada porque los archivos son pequeños: el ZIP agrega un encabezado por cada archivo (nombre, fecha, CRC), que pesa bastante en comparación con 681 bytes. Aun así, contactos.dat comprime bien porque los campos están rellenos de espacios, que se repiten mucho. Con miles de contactos, el encabezado pesaría proporcionalmente menos y el porcentaje de reducción sería mayor.

6. Funcionamiento de la aplicación

6.1 Contactos: agregar, buscar y modificar

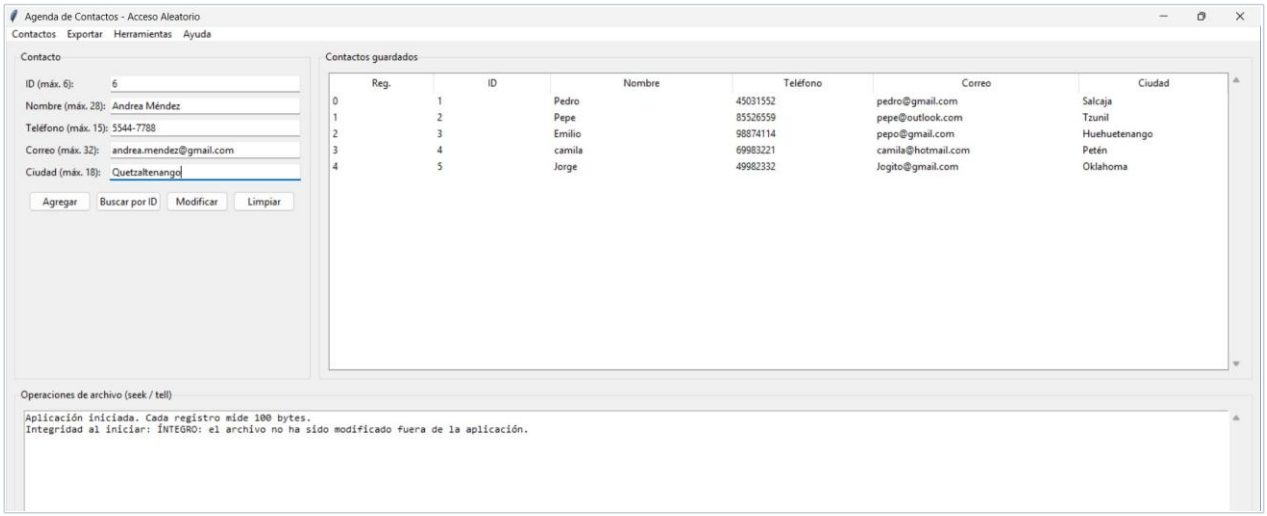


Figura 12. Formulario con los datos de un contacto nuevo (id 6) antes de presionar Agregar.

Las figuras 3, 4 y 5 de las secciones anteriores muestran el resultado de agregar, modificar y buscar, junto con el recorrido de seek() y tell() en la bitácora.

6.2 Exportar a JSON y reporte PDF

La opción Exportar a JSON lee todos los registros de contactos.dat y los guarda en **contactos_exportados.json** junto con la fecha de exportación. El reporte PDF se genera **a partir de ese JSON**, con una tabla de contactos y la fecha de generación.

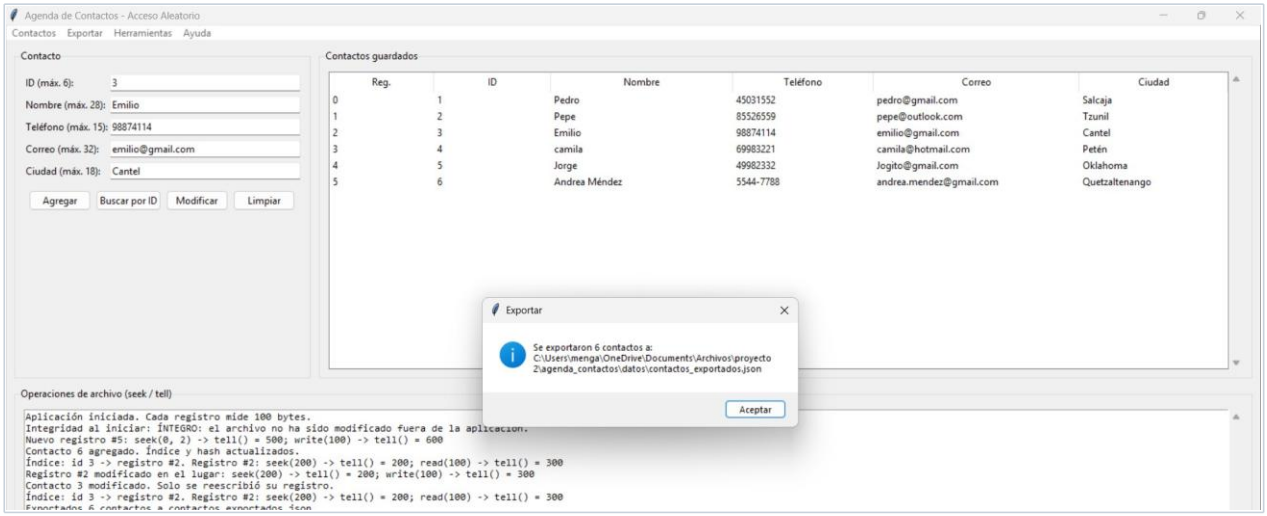


Figura 13. Exportación de los 6 contactos a contactos_exportados.json.

```
{
    "fecha_exportacion": "08/11/2026 23:50:43",
    "total_contactos": 6,
    "contactos": [
        {
            "id": 1,
            "nombre": "Pedro",
            "telefono": "45031552",
            "correo": "pedro@gmail.com",
            "ciudad": "Salcaja"
        },
        ...
```

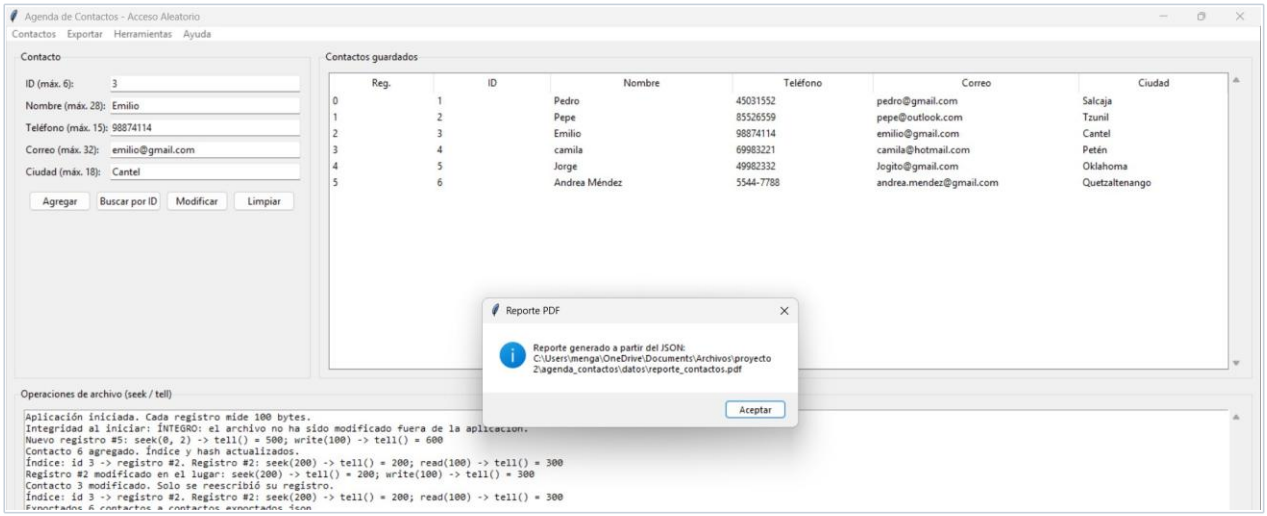


Figura 14. Generación del reporte PDF a partir del JSON.



Figura 15. Reporte PDF generado por la aplicación (reporte_contactos.pdf).

6.3 Información del archivo

Esta ventana usa **os.stat()** para el tamaño, la fecha de última modificación y los permisos, y **os.access()** para comprobar si hay permiso de lectura y de escritura. En contactos.dat también

muestra cuántos registros tiene (tamaño ÷ 100).

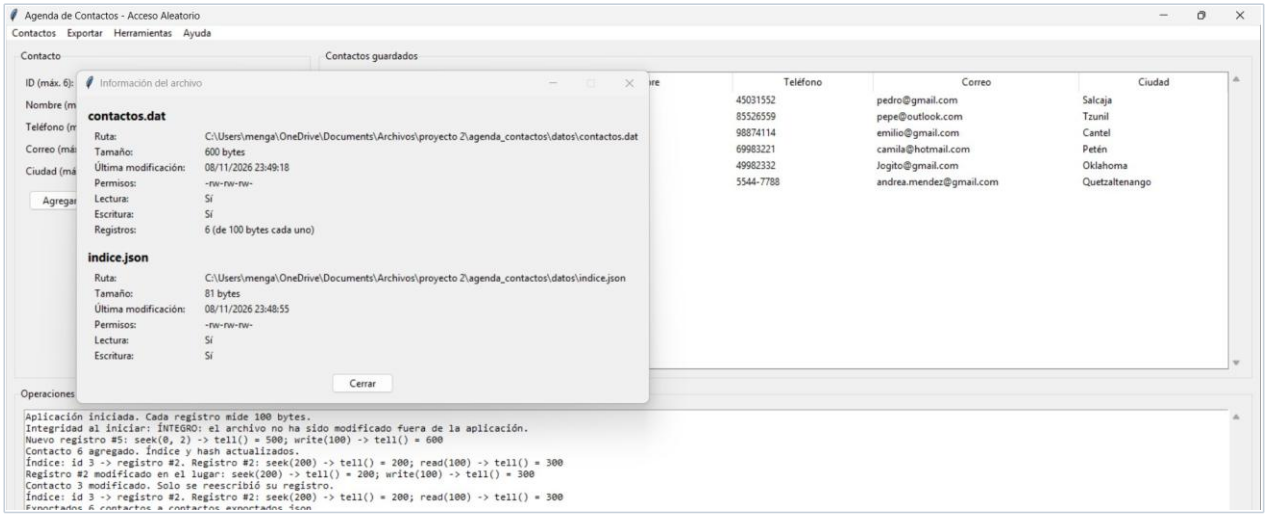


Figura 16. Información del archivo: contactos.dat (600 bytes, 6 registros) e indice.json (81 bytes).

6.4 Manejo de errores

Las operaciones se ejecutan dentro de bloques try/except, así que la aplicación no se cierra ante un problema con los archivos:

- Si **contactos.dat** no existe, la agenda inicia vacía; si su tamaño no es múltiplo de 100, avisa que puede estar corrupto.
- Si **indice.json** no existe, está corrupto o no coincide con contactos.dat, se reconstruye recorriendo el archivo de contactos.
- Si no hay permisos de lectura o escritura (PermissionError), se muestra un mensaje.
- Si se busca un id que no existe, se avisa (figura 6).
- Se valida que el id sea un entero positivo y único, que ningún campo esté vacío, que el teléfono y el correo tengan un formato válido y que cada dato quepa en su campo.
- Todos los archivos se leen y escriben en UTF-8.

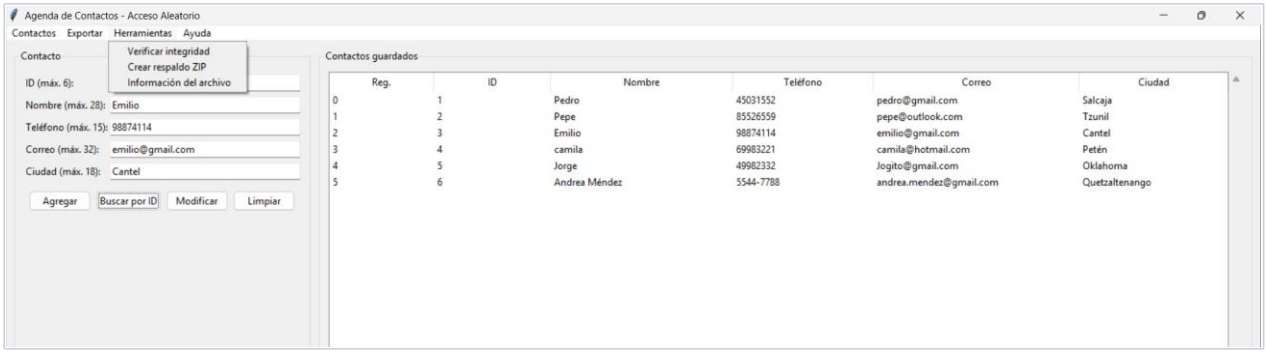


Figura 17. Menú Herramientas: verificar integridad, crear respaldo ZIP e información del archivo.

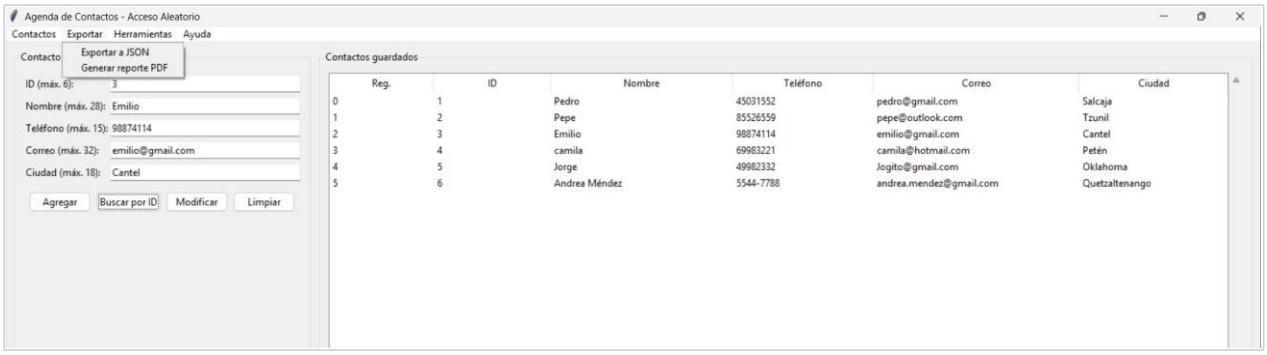


Figura 18. Menú Exportar: exportar a JSON y generar reporte PDF.

Conclusiones

- Usar registros de tamaño fijo permite calcular la posición de cualquier contacto (número de registro × 100) y llegar a él con seek() sin leer los registros anteriores.
- La modificación en el lugar con el modo "rb+" reescribe solo los 100 bytes del registro editado; el resto del archivo no se toca.
- El índice en JSON convierte la búsqueda por id en una sola lectura, sin importar cuántos contactos haya, y permite saber que un id no existe sin abrir el archivo.
- El hash SHA-256 detectó un cambio de una sola palabra hecho fuera de la aplicación, aunque el tamaño del archivo no cambió.
- El respaldo ZIP redujo los archivos de 681 a 501 bytes (26.43 %); con más contactos la reducción sería mayor.
- La desventaja de los registros fijos es que se desperdicia espacio en los campos cortos; a cambio, se obtiene acceso directo y modificacion**es en el lugar**.